BRACT's

Vishwakarma Institute of Technology, Pune Practical Implementation Sheet

Department: CSE-AI Semester: 1 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 10 Roll no: 51 PRN No: 12412161

Subject Name : Deep Learning

Name of Student: Utkarsh Salunkhe

Problem Statement :

Design and implement a deep learning application for audio classification that incorporates Speech-to-Text (STT) and Text-to-Speech (TTS) using an appropriate deep learning architecture, and evaluate its performance using suitable metrics.

Algorithm:

Input: Audio dataset containing labeled spoken commands and a user-recorded audio file.

Output: Predicted audio class, speech transcription, synthesized speech, and model evaluation metrics.

1. Start.
2. Import the required libraries, including TensorFlow, NumPy, Librosa, SpeechRecognition, and gTTS.
3. Download and extract the Mini Speech Commands dataset.
4. Validate the dataset and remove invalid audio files.
5. Divide the dataset into training (70%), validation (15%), and testing (15%) sets.
6. Preprocess audio:
    - Convert audio to mono format.
    - Resample audio to 16 kHz.
    - Pad or truncate each audio signal to one second.
7. Extract features: Apply the Short-Time Fourier Transform (STFT) to generate spectrograms from the audio signals.
8. Build the CNN model:
    - Add convolutional layers with ReLU activation.
    - Apply batch normalization and max pooling.
    - Add global average pooling and dropout.
    - Add a softmax output layer for class prediction.
9. Compile the model using the Adam optimizer, sparse categorical cross-entropy loss, and accuracy metric.
10. Train the CNN using the training dataset and monitor performance on the validation dataset. Apply early stopping to reduce overfitting.
11. Evaluate the trained model on the test dataset.

12. Calculate accuracy, macro-precision, macro-recall, macro-F1-score, and the confusion matrix.
13. Accept a user-recorded audio file as input.
14. Perform STT: Convert the input speech into text using the speech recognition service.
15. Perform audio classification: Preprocess the audio, generate its spectrogram, and pass it through the trained CNN to predict the command class.
16. Display the recognized text, predicted class, and classification confidence.
17. Perform TTS: Convert the predicted command or response into speech using gTTS and play the generated audio.
18. Save the trained model, evaluation metrics, and performance plots.
19. Stop.

Flowchart

Start

Download and validate audio dataset

Preprocess audio and extract spectrograms

Build and train CNN model

Evaluate using classification metrics

Accept user audio input

Speech-to-Text (STT)

CNN audio classification

Display transcription and predicted class

Generate spoken response using TTS

Stop

Code:

```
# ============================================================
# DEEP LEARNING AUDIO CLASSIFICATION WITH STT AND TTS
# CNN + TensorFlow + SpeechRecognition + gTTS
# Google Colab: Run this entire cell
# ============================================================
```

```
import os
import sys
import time
import random
import zipfile
import pathlib
import subprocess
import warnings
warnings.filterwarnings("ignore")

# -------------------- 1. INSTALL LIBRARIES --------------------
subprocess.check_call([
    sys.executable, "-m", "pip", "install", "-q",
    "SpeechRecognition", "gTTS", "librosa",
    "scikit-learn", "seaborn"
])

import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns
import librosa
import speech_recognition as sr

from gtts import gTTS
from IPython.display import Audio, display
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
```

```
    f1_score, classification_report, confusion_matrix
)

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))

# -------------------- 2. DOWNLOAD DATASET --------------------
DATA_ROOT = pathlib.Path("/content/audio_project_data")
DATA_ROOT.mkdir(parents=True, exist_ok=True)

ZIP_PATH = DATA_ROOT / "mini_speech_commands.zip"
EXTRACT_ROOT = DATA_ROOT / "extracted"
DATASET_PATH = EXTRACT_ROOT / "mini_speech_commands"

URL = (
    "https://storage.googleapis.com/"
    "download.tensorflow.org/data/mini_speech_commands.zip"
)

if not ZIP_PATH.exists():
    print("\nDownloading dataset...")
    tf.keras.utils.get_file(
        fname="mini_speech_commands.zip",
        origin=URL,
```

```
        cache_dir=str(DATA_ROOT / "cache"),
        cache_subdir=""
    )
    cached = DATA_ROOT / "cache" / "mini_speech_commands.zip"
    if cached.exists() and not ZIP_PATH.exists():
        import shutil
        shutil.copy2(cached, ZIP_PATH)

# Download fallback if the cache filename differs.
if not ZIP_PATH.exists():
    import urllib.request
    urllib.request.urlretrieve(URL, ZIP_PATH)

print("Extracting dataset...")
if not DATASET_PATH.exists():
    with zipfile.ZipFile(ZIP_PATH, "r") as archive:
        archive.extractall(EXTRACT_ROOT)

if not DATASET_PATH.exists():
    candidates = list(EXTRACT_ROOT.rglob("mini_speech_commands"))
    if not candidates:
        raise FileNotFoundError(
            "Dataset folder not found after extraction."
        )
    DATASET_PATH = candidates[0]

# -------------------- 3. VALIDATE DATASET --------------------
# Retain only genuine, readable WAV files inside class folders.
import wave
```

```
class_dirs = [
    p for p in DATASET_PATH.iterdir()
    if p.is_dir() and not p.name.startswith(".")
]

if not class_dirs:
    raise RuntimeError("No class directories found in the dataset.")

bad_files = []
valid_counts = {}

for folder in class_dirs:
    count = 0
    for file in folder.iterdir():
        if not file.is_file() or file.suffix.lower() != ".wav":
            continue
        try:
            with wave.open(str(file), "rb") as wav:
                if wav.getnframes() <= 0:
                    raise ValueError("Empty WAV file")
            count += 1
        except Exception:
            bad_files.append(file)
    valid_counts[folder.name] = count

for file in bad_files:
    try:
        file.unlink()
```

```
    except OSError:
        pass

print("\nDataset:", DATASET_PATH)
print("Classes and valid WAV counts:", valid_counts)
print("Invalid files removed:", len(bad_files))

CLASS_NAMES = sorted(
    p.name for p in DATASET_PATH.iterdir()
    if p.is_dir() and any(p.glob("*.wav"))
)
NUM_CLASSES = len(CLASS_NAMES)

if NUM_CLASSES < 2:
    raise RuntimeError("Not enough valid audio classes were found.")

print("Classes:", CLASS_NAMES)

# -------------------- 4. LOAD AND SPLIT DATA --------------------
# Use file paths for explicit, reproducible train/validation/test splits.
records = []

for label_id, class_name in enumerate(CLASS_NAMES):
    files = sorted((DATASET_PATH / class_name).glob("*.wav"))
    files = [str(f) for f in files]
    random.Random(SEED).shuffle(files)

    n = len(files)
```

```
    n_train = int(n * 0.70)
    n_val = int(n * 0.15)

    for f in files[:n_train]:
        records.append((f, label_id, "train"))
    for f in files[n_train:n_train + n_val]:
        records.append((f, label_id, "val"))
    for f in files[n_train + n_val:]:
        records.append((f, label_id, "test"))

split_df = pd.DataFrame(
    records, columns=["path", "label", "split"]
)

print("\nSplit counts:")
print(split_df.groupby(["split",
"label"]).size().unstack(fill_value=0))

# -------------------- 5. AUDIO PREPROCESSING --------------------
SAMPLE_RATE = 16000
AUDIO_LENGTH = 16000
FRAME_LENGTH = 255
FRAME_STEP = 128
FFT_LENGTH = 256
BATCH_SIZE = 64

def load_waveform(path):
    audio, _ = librosa.load(
        path, sr=SAMPLE_RATE, mono=True
```

```
    )
    if len(audio) < AUDIO_LENGTH:
        audio = np.pad(
            audio, (0, AUDIO_LENGTH - len(audio))
        )
    else:
        audio = audio[:AUDIO_LENGTH]
    return audio.astype(np.float32)


def make_spectrogram(waveform):
    stft = tf.signal.stft(
        waveform,
        frame_length=FRAME_LENGTH,
        frame_step=FRAME_STEP,
        fft_length=FFT_LENGTH
    )
    return tf.math.log(tf.abs(stft) + 1e-6)[..., tf.newaxis]


def load_example(path, label):
    audio = tf.numpy_function(
        load_waveform, [path], tf.float32
    )
    audio.set_shape([AUDIO_LENGTH])


    spec = make_spectrogram(audio)
    spec.set_shape([
        (AUDIO_LENGTH - FRAME_LENGTH) // FRAME_STEP + 1,
        FFT_LENGTH // 2 + 1,
        1
```

```
    ])
    return spec, label


def build_tf_dataset(split, training=False):
    subset = split_df[split_df["split"] == split]
    paths = subset["path"].values
    labels = subset["label"].values.astype(np.int32)

    ds = tf.data.Dataset.from_tensor_slices((paths, labels))
    if training:
        ds = ds.shuffle(
            len(paths), seed=SEED,
            reshuffle_each_iteration=True
        )

    ds = ds.map(
        load_example,
        num_parallel_calls=tf.data.AUTOTUNE
    )
    ds = ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
    return ds


train_ds = build_tf_dataset("train", training=True)
val_ds = build_tf_dataset("val")
test_ds = build_tf_dataset("test")


for sample_x, sample_y in train_ds.take(1):
    INPUT_SHAPE = tuple(sample_x.shape[1:])
```

```
print("\nModel input shape:", INPUT_SHAPE)


# -------------------- 6. VISUALIZE SPECTROGRAM --------------------
sample_spec, sample_label = next(iter(train_ds))
plt.figure(figsize=(9, 4))
plt.imshow(
    sample_spec[0, :, :, 0].numpy().T,
    origin="lower", aspect="auto"
)
plt.title("Example Audio Spectrogram")
plt.xlabel("Time Frames")
plt.ylabel("Frequency Bins")
plt.tight_layout()
plt.show()


# -------------------- 7. BUILD CNN --------------------
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=INPUT_SHAPE),


    tf.keras.layers.Conv2D(
        32, 3, padding="same", activation="relu"
    ),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(2),


    tf.keras.layers.Conv2D(
        64, 3, padding="same", activation="relu"
    ),
    tf.keras.layers.BatchNormalization(),
```

```
    tf.keras.layers.MaxPooling2D(2),

    tf.keras.layers.Conv2D(
        128, 3, padding="same", activation="relu"
    ),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.GlobalAveragePooling2D(),

    tf.keras.layers.Dropout(0.35),
    tf.keras.layers.Dense(NUM_CLASSES, activation="softmax")
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

# -------------------- 8. TRAIN MODEL --------------------
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=4,
        restore_best_weights=True
    ),
    tf.keras.callbacks.ModelCheckpoint(
        "/content/best_audio_classifier.keras",
```

```
        monitor="val_loss",
        save_best_only=True
    )
]

print("\nTraining CNN...")
start_time = time.time()

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20,
    callbacks=callbacks,
    verbose=1
)

training_seconds = time.time() - start_time
print(f"Training time: {training_seconds:.2f} seconds")

# -------------------- 9. PLOT TRAINING CURVES --------------------
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Train")
plt.plot(history.history["val_accuracy"], label="Validation")
plt.title("Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
```

```
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Train")
plt.plot(history.history["val_loss"], label="Validation")
plt.title("Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.savefig("/content/training_curves.png", dpi=150)
plt.show()

# -------------------- 10. EVALUATE TEST DATA --------------------
y_true = []
y_pred = []

for x_batch, y_batch in test_ds:
    probabilities = model.predict(x_batch, verbose=0)
    y_pred.extend(np.argmax(probabilities, axis=1))
    y_true.extend(y_batch.numpy())

y_true = np.asarray(y_true)
y_pred = np.asarray(y_pred)

accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(
    y_true, y_pred, average="macro", zero_division=0
)
```

```
recall = recall_score(
    y_true, y_pred, average="macro", zero_division=0
)
f1 = f1_score(
    y_true, y_pred, average="macro", zero_division=0
)


print("\n========== TEST RESULTS ==========")
print(f"Test Accuracy : {accuracy:.4f} ({accuracy*100:.2f}%)")
print(f"Macro Precision: {precision:.4f}")
print(f"Macro Recall   : {recall:.4f}")
print(f"Macro F1-score : {f1:.4f}")
print(f"Training Time  : {training_seconds:.2f} seconds")


print("\nClassification Report:")
print(classification_report(
    y_true,
    y_pred,
    labels=np.arange(NUM_CLASSES),
    target_names=CLASS_NAMES,
    zero_division=0
))


metrics_df = pd.DataFrame({
    "Metric": [
        "Test Accuracy", "Macro Precision",
        "Macro Recall", "Macro F1-score",
        "Training Time (seconds)"
    ],
```

```
    "Value": [
        accuracy, precision, recall, f1, training_seconds
    ]
})
metrics_df.to_csv("/content/evaluation_results.csv", index=False)
print(metrics_df.to_string(index=False))


# -------------------- 11. CONFUSION MATRIX --------------------
cm = confusion_matrix(
    y_true, y_pred, labels=np.arange(NUM_CLASSES)
)


plt.figure(figsize=(9, 7))
sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues",
    xticklabels=CLASS_NAMES,
    yticklabels=CLASS_NAMES
)
plt.title("Audio Classification Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.tight_layout()
plt.savefig("/content/confusion_matrix.png", dpi=150)
plt.show()


# -------------------- 12. SAVE MODEL --------------------
model.save("/content/audio_classification_model.keras")
print("Model saved: /content/audio_classification_model.keras")
```

```python
# -------------------- 13. STT --------------------
def speech_to_text(audio_path, language="en-IN"):
    recognizer = sr.Recognizer()
    try:
        with sr.AudioFile(audio_path) as source:
            audio_data = recognizer.record(source)

        text = recognizer.recognize_google(
            audio_data, language=language
        )
        print("STT transcription:", text)
        return text

    except sr.UnknownValueError:
        print("STT could not understand the audio.")
        return ""

    except sr.RequestError as e:
        print("STT service error:", e)
        return ""

# -------------------- 14. TTS --------------------
def text_to_speech(text, output_path="/content/speech_output.mp3"):
    if not text or not text.strip():
        print("No text to synthesize.")
        return

    try:
        gTTS(text=text, lang="en", slow=False).save(output_path)
```

```
        print("TTS saved:", output_path)
        display(Audio(output_path))
    except Exception as e:
        print("TTS error:", e)

# -------------------- 15. CLASSIFY AN AUDIO FILE --------------------
def classify_audio(audio_path):
    waveform = load_waveform(audio_path)
    spec = make_spectrogram(
        tf.convert_to_tensor(waveform, dtype=tf.float32)
    )
    spec = spec[tf.newaxis, ...]

    probabilities = model.predict(spec, verbose=0)[0]
    idx = int(np.argmax(probabilities))

    result = {
        "predicted_class": CLASS_NAMES[idx],
        "confidence": float(probabilities[idx]),
        "probabilities": {
            name: float(probabilities[i])
            for i, name in enumerate(CLASS_NAMES)
        }
    }

    print("\nPredicted command:", result["predicted_class"])
    print(f"Model confidence: {result['confidence']:.2%}")
    return result
```

```
# -------------------- 16. UPLOAD AUDIO AND RUN DEMO ------------------
--
# Upload a WAV recording containing one of the trained commands.
# Use a short, clear recording for best results.

from google.colab import files

print("\nUpload a WAV audio file to test STT and CNN classification.")
uploaded = files.upload()

if uploaded:
    test_audio_path = next(iter(uploaded.keys()))

    # CNN classification
    result = classify_audio(test_audio_path)

    # STT transcription
    print("\n========== SPEECH-TO-TEXT ==========")
    transcript = speech_to_text(test_audio_path)

    # Spoken output
    print("\n========== TEXT-TO-SPEECH ==========")
    response = (
        f"The detected command is {result['predicted_class']}."
    )
    text_to_speech(response)

    print("\n========== FINAL SUMMARY ==========")
    print("Transcription:", transcript or "Unavailable")
```

```
    print("Predicted class:", result["predicted_class"])
    print(f"Confidence: {result['confidence']:.2%}")


# -------------------- 17. DOWNLOAD REPORT FILES --------------------
print("\nGenerated files:")
for p in [
    "/content/audio_classification_model.keras",
    "/content/evaluation_results.csv",
    "/content/training_curves.png",
    "/content/confusion_matrix.png",
    "/content/speech_output.mp3"
]:
    if os.path.exists(p):
        print(p)



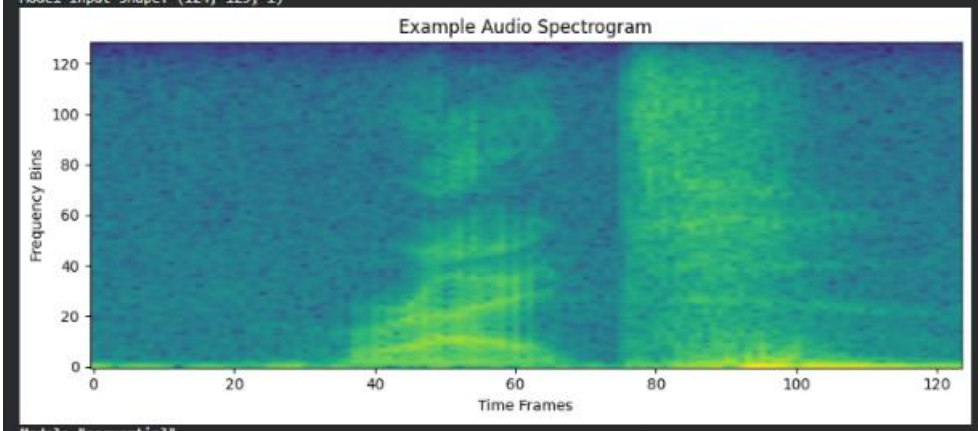
print("\nPROJECT EXECUTION COMPLETED.")
```

Output:

```
Dataset: /content/audio_project_data/extracted/mini_speech_commands
Classes and valid WAV counts: {'yes': 1000, 'right': 1000, 'go': 1000, 'up': 1000, 'left': 1000, 'no': 1000, 'down': 1000, 'stop': 1000}
Invalid files removed: 0
Classes: ['down', 'go', 'left', 'no', 'right', 'stop', 'up', 'yes']

Split counts:
label    0    1    2    3    4    5    6    7
split
test   150  150  150  150  150  150  150  150
train  700  700  700  700  700  700  700  700
val    150  150  150  150  150  150  150  150

Model input shape: (124, 129, 1)
```

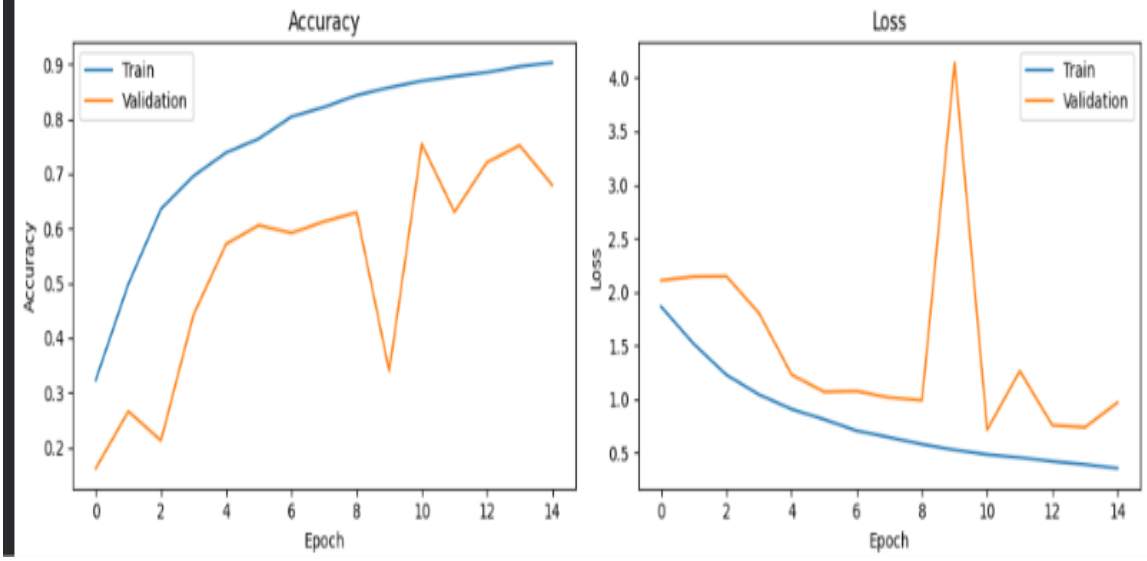



```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 124, 129, 32)	320
batch_normalization (BatchNormalization)	(None, 124, 129, 32)	128
max_pooling2d (MaxPooling2D)	(None, 62, 64, 32)	0
conv2d_1 (Conv2D)	(None, 62, 64, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 62, 64, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 31, 32, 64)	0
conv2d_2 (Conv2D)	(None, 31, 32, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 31, 32, 128)	512
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0
dropout (Dropout)	(None, 128)	0
dense (Dense)	(None, 8)	1,032

```
Total params: 94,600 (369.53 KB)
Trainable params: 94,152 (367.78 KB)
Non-trainable params: 448 (1.75 KB)
```

Training time: 141.72 seconds



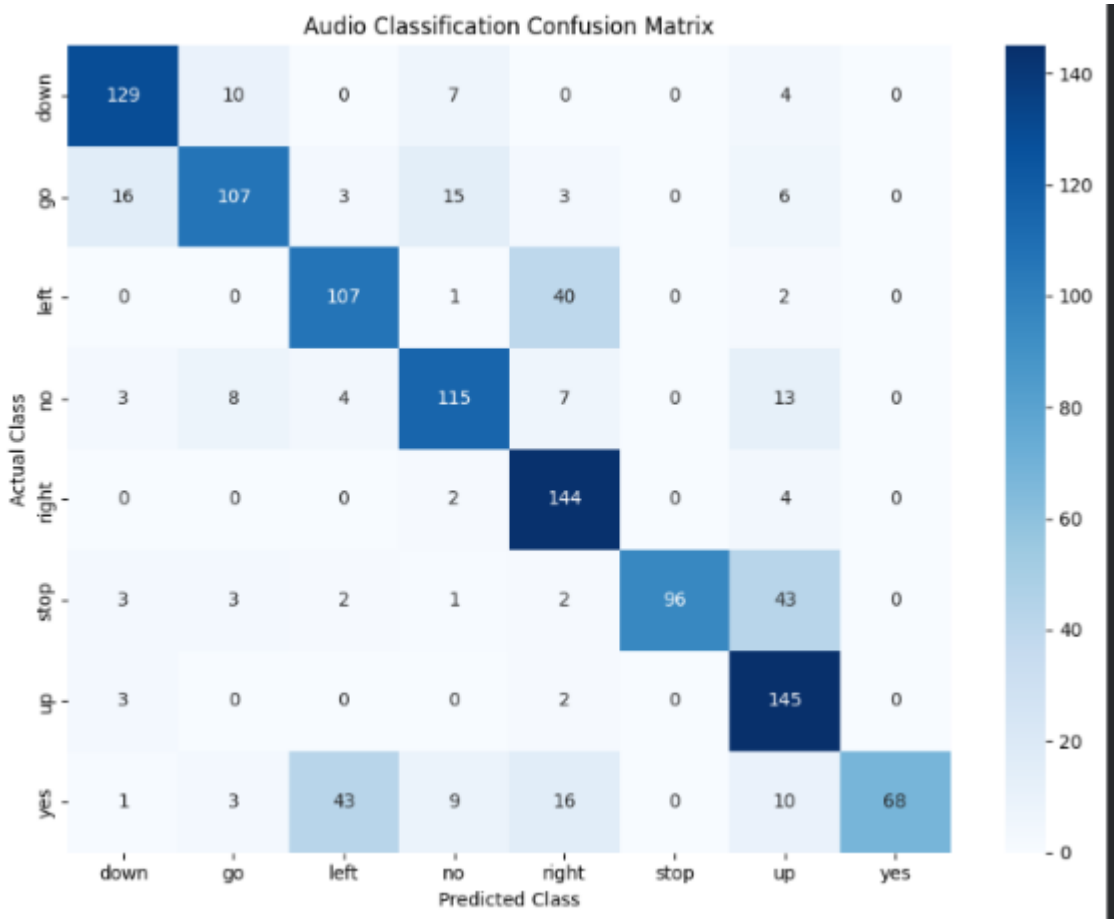

```
========== TEST RESULTS ==========
Test Accuracy : 0.7592 (75.92%)
Macro Precision: 0.8000
Macro Recall   : 0.7592
Macro F1-score : 0.7539
Training Time  : 141.72 seconds

Classification Report:
              precision    recall  f1-score   support

        down       0.83      0.86      0.85       150
          go       0.82      0.71      0.76       150
        left       0.67      0.71      0.69       150
          no       0.77      0.77      0.77       150
       right       0.67      0.96      0.79       150
        stop       1.00      0.64      0.78       150
          up       0.64      0.97      0.77       150
         yes       1.00      0.45      0.62       150

    accuracy                           0.76      1200
   macro avg       0.80      0.76      0.75      1200
weighted avg       0.80      0.76      0.75      1200

                Metric      Value
         Test Accuracy   0.759167
        Macro Precision  0.800042
           Macro Recall  0.759167
       Macro F1-score    0.753934
Training Time (seconds) 141.718767
```


Audio Classification Confusion Matrix
Actual Class
Predicted Class
down
go
left
no
right
stop
up
yes
129 10 0 7 0 0 4 0
16 107 3 15 3 0 6 0
0 0 107 1 40 0 2 0
3 8 4 115 7 0 13 0
0 0 0 2 144 0 4 0
3 3 2 1 2 96 43 0
3 0 0 0 2 0 145 0
1 3 43 9 16 0 10 68
140
120
100
80
60
40
20
0